

Git 기본 명령어

git init - (로컬)저장소 만들기

```
$ mkdir sample # 디렉토리 생성
$ cd sample # 디렉토리 이동
$ touch red orange # 파일 만들기
$ echo "빨강" >> red # 파일 수정하기
$ echo "주황" >> orange # 파일 수정하기

$ git init # (로컬)저장소 만들기
$ ls -al # 디렉토리 생성
```

```
good593 ❤️ ➤ ~/dev/sample ➤ master
▶ ls -al
total 16
drwxr-xr-x  5 gyoungwon-cho staff 160 12 20 09:16 .
drwxr-xr-x 12 gyoungwon-cho staff 384 12 20 09:15 ..
drwxr-xr-x  9 gyoungwon-cho staff 288 12 20 09:16 .git
-rw-r--r--  1 gyoungwon-cho staff   7 12 20 09:15 orange
-rw-r--r--  1 gyoungwon-cho staff   7 12 20 09:15 red
good593 ❤️ ➤ ~/dev/sample ➤ master
▶ █
```

git status - 현재 상태 확인

현재 작업 중인 파일의 상태를 확인합니다.

```
$ git status
```

```
good593 ❤️ ~/dev/sample master
```

```
▶ git status
```

```
현재 브랜치 master
```

```
아직 커밋이 없습니다
```

```
추적하지 않는 파일:
```

```
(커밋할 사항에 포함하려면 "git add <파일>..."을 사용하십시오)
```

```
orange
```

```
red
```

```
커밋할 사항을 추가하지 않았지만 추적하지 않는 파일이 있습니다 (추적하려면 "git add"를 사용하십시오)
```

```
good593 ❤️ ~/dev/sample master
```

```
▶
```

git add - 현재 상태 추적

파일의 변경사항을 인덱스index에 추가합니다. Git은 커밋하기 전, 인덱스에 먼저 커밋할 파일을 추가합니다.

```
$ git add -A # all이라는 뜻으로 새로운 파일들 전체 추적
$ git status
```

```
good593 ♥ ~/dev/sample ➤ master
git add -A
good593 ♥ ~/dev/sample ➤ master +
git status
현재 브랜치 master

아직 커밋이 없습니다

커밋할 변경 사항:
(스테이지 해제하려면 "git rm --cached <파일>..."을 사용하십시오)
    새 파일:      orange
    새 파일:      red

good593 ♥ ~/dev/sample ➤ master +
```

git commit - 현재 상태 저장

인덱스에 추가된 변경 사항을 이력에 추가합니다.

```
$ git commit -m "v1 commit" # v1 commit이라는 메시지와 함께 상태 저장  
$ git log # 커밋 히스토리 조회(화면 나가기: q)
```

```
good593 🍷 ~/dev/sample ➤ master +  
➤ git commit -m "v1 commit"  
[master (최상위-커밋) ca5e918] v1 commit  
2 files changed, 2 insertions(+)  
create mode 100644 orange  
create mode 100644 red  
good593 🍷 ~/dev/sample ➤ master  
➤ git log  
good593 🏰 ~/dev/sample ➤ master  
➤
```

```
commit ca5e918d95c3c0459b3cb274a7b58f3d81f3d39c (HEAD -> master)
```

```
Author: good593 <goodwon593@gmail.com>
```

```
Date:   Wed Dec 20 09:59:07 2023 +0900
```

```
    v1 commit
```

```
(END)
```

현재 Git 저장소 이력



첫번째 커밋

git log - 커밋 히스토리 조회

- 새 파일 추가 및 커밋
- log화면 나가기: q

```
# 새 파일 추가
$ touch yellow
$ echo "노랑" >> yellow
$ git status

# 새 파일 커밋
$ git add -A
$ git commit -m "v2 commit"
$ git log # 2개의 커밋 이력이 조회됨
```

```
commit 4bc138dd66f51052a68aa217d510cdf8bac974ac (HEAD -> master)
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 10:12:08 2023 +0900

    v2 commit

commit ca5e918d95c3c0459b3cb274a7b58f3d81f3d39c
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 09:59:07 2023 +0900

    v1 commit
(END)
```

현재 Git 저장소 이력



다양한 변화(시도)

작업

1. red 삭제
2. orange에 내용 추가
3. green 파일 추가
4. 상태 확인
5. 전체 파일 인덱스에 추가
6. 세 번째 이력 커밋
7. 커밋 이력 조회

```
$ rm red # 1.  
$ echo "오렌지" >> orange # 2.  
$ touch green # 3.  
$ git status # 4.
```

```
commit 369e3b6523c8d382315612aaf58442282e0f3bc4 (HEAD -> master)
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 10:15:03 2023 +0900

    v3 commit

commit 4bc138dd66f51052a68aa217d510cdf8bac974ac
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 10:12:08 2023 +0900

    v2 commit

commit ca5e918d95c3c0459b3cb274a7b58f3d81f3d39c
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 09:59:07 2023 +0900

    v1 commit
(END)
```

현재 Git 저장소 이력



git reset - 이전 상태로(이력제거)

- 특정 커밋까지 이력을 초기화합니다. 바로 전, 또는 n번 전까지 작업했던 내용을 취소할 수 있습니다.
- 열심히 작업했는데, 전혀 엉뚱한 걸 했거나 작업한 내용이 필요 없어질 때 사용합니다.
- 이력이 지워지기 때문에 주의해야 합니다.

작업

1. 커밋 이력(ID) 조회
2. 2번 커밋으로 이력 초기화(3번 이력 삭제)
3. 커밋 이력 조회(3번 이력 삭제 확인)
 - 지웠던 `red` 파일 이 되살아나고 `orange` 파일 내용이 수정되고,
 - `green` 파일 이 사라졌다.

```
$ git log
$ git reset {v2 커밋 아이디} --hard # 커밋 아이디 예) 27a00b7 (앞에 7자 정도 복사)
$ git log # 3번 이력이 삭제됨...
```

```
good593 👑 > ~/dev/sample > master
▶ git log
good593 👑 > ~/dev/sample > master
▶ git reset 4bc138dd66f51052a68aa217d510cdf8bac974ac
리셋 뒤에 스테이징하지 않은 변경 사항:
M      orange
D      red
good593 👑 > ~/dev/sample > master ±
▶
```

```
commit 4bc138dd66f51052a68aa217d510cdf8bac974ac (HEAD -> master)
```

```
Author: good593 <goodwon593@gmail.com>
```

```
Date: Wed Dec 20 10:12:08 2023 +0900
```

```
v2 commit
```

```
commit ca5e918d95c3c0459b3cb274a7b58f3d81f3d39c
```

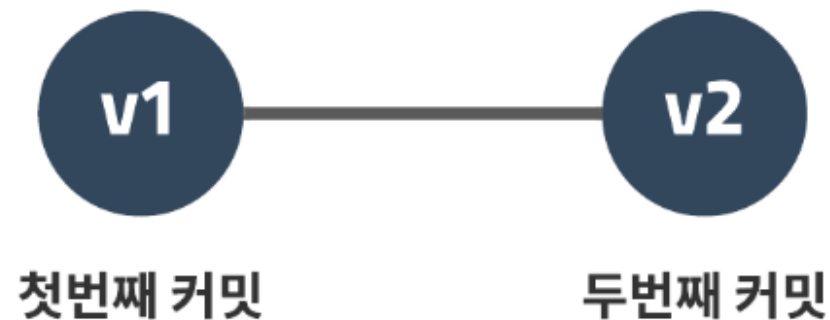
```
Author: good593 <goodwon593@gmail.com>
```

```
Date: Wed Dec 20 09:59:07 2023 +0900
```

```
v1 commit
```

```
(END)
```

현재 Git 저장소 이력



git revert - 이전 상태로 (이력 유지)

- 특정 커밋을 취소하는 새로운 커밋을 만듭니다. 여기선 3번 커밋을 취소하는 새로운 커밋을 생성하여 마치 2번 커밋 상태로 돌아간 것 같지만 기존 이력을 유지하는 모습을 확인합니다.
- 일반적으로 특정 버전을 배포했는데 문제가 생기면 문제가 생긴 커밋을 revert합니다. (빠른 조치/롤백) 다시 원복한 상태로 작업을 이어서 하고 해당 문제를 수정하면 다시 커밋하는 방식을 사용합니다.

작업

1. git reset 명령어로 3번 커밋이 지워졌다면 이전 실습을 통해 다시 커밋 추가
2. git log로 3번 커밋 ID 조회
3. 3번 커밋 취소
 - 지웠던 red가 되살아나고 orange 내용이 수정되고 green 파일이 사라진 것을 확인
 - git log를 통해 새로운 커밋이 추가된 것을 확인

```
$ git add -A
$ git commit -m "v4 commit"
$ git log
$ git revert --no-commit {v4 커밋 아이디} # 커밋 아이디 예) 306b947 (앞에 7자 정도 복사)
$ git commit -m "new revert"
$ git log
```

```
good593 👑 > ~/dev/sample > master +
> git revert --no-commit e823044d4c1959b047f25b54ad777dfafba6debf
good593 👑 > ~/dev/sample > master +
> git commit -m "new revert"
[master 1a1beb6] new revert
3 files changed, 1 insertion(+), 1 deletion(-)
delete mode 100644 green
create mode 100644 red
good593 👑 > ~/dev/sample > master
> git log
good593 👑 > ~/dev/sample > master
> █
```

```
commit 1a1beb6cc21a1d5efaa092be3b66668b1afad296 (HEAD -> master)
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 10:30:14 2023 +0900

    new revert

commit e823044d4c1959b047f25b54ad777dfafba6debf
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 10:22:48 2023 +0900

    v4 commit

commit 4bc138dd66f51052a68aa217d510cdf8bac974ac
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 10:12:08 2023 +0900

    v2 commit

commit ca5e918d95c3c0459b3cb274a7b58f3d81f3d39c
Author: good593 <goodwon593@gmail.com>
Date:   Wed Dec 20 09:59:07 2023 +0900

    v1 commit
(END)
```

현재 Git 저장소 이력

